S.O.L.I.D

- Single responsibility Principle
- open-closed principle
- Liskov substitution principle
- Interface segregation principle
- Dependancy inversion Principle.

SRP (SOC)

↳ separation of concern

"idea" - a class should have its primary responsibility, and it should not take any other resp.

```
class Journal:

    def __init__(self):
        self.entry = []
        self.count = 0

    def add_entry(self, text):
        pass

    def remove_entry(self, pos):
        pass

    def __str__(self):
        pass
```

resp of journaling

These 3 belong specifically to journal, as they perform actions needed in a journal entry process

```
    def save_to_file(self, filename):
        pass

    def load_from_file(self, filename):
        pass
```

resp of persistence

these can be repetitive in other classes also

Correct approach

```
Class Journal:

    def __init__(self):
        self.entries = []
        self.counter = 0

    def add_entry(self, text):
        pass

    def remove_entry(self, pos):
        del self.entries[pas]

    def __str__(self):
        print(self.entries)
```

```
Class PersistenceManager:

    @staticmethod
    def load(filename):
        pass

    @staticmethod
    def save(journal, filename):
        pass
```

Anti pattern of SRP (SOC):- god_object, where all functionalities are put under a single class.

Open-closed Pattern

idea - "when you add new functionality, you add via extension, not via modification"

```
class ProductFilter:

    def filter_by_color(self, products, color):
        for p in products:
            if p.color == color: yield p

    def filter_by_size(self, products, size):
        for p in products:
            if p.size == size: yield p
```

open pattern

Correct approach

```
class Specification:
    def __init__(self, items):
        pass

class Filter:
    def filter(self, items, spec):
        pass
```

Base classes.

```
class ColorSpecification(Specification):
    def __init__(self, color):
        self.color = color

    def is_satisfied(self, item):
        return item.color == self.color
```

```
class BetterFilter(Filter):

    def filter(self, items, spec):
        for item in items:
            if spec.is_satisfied(item):
                yield item
```

for multiple filters, we need to make a combinator.

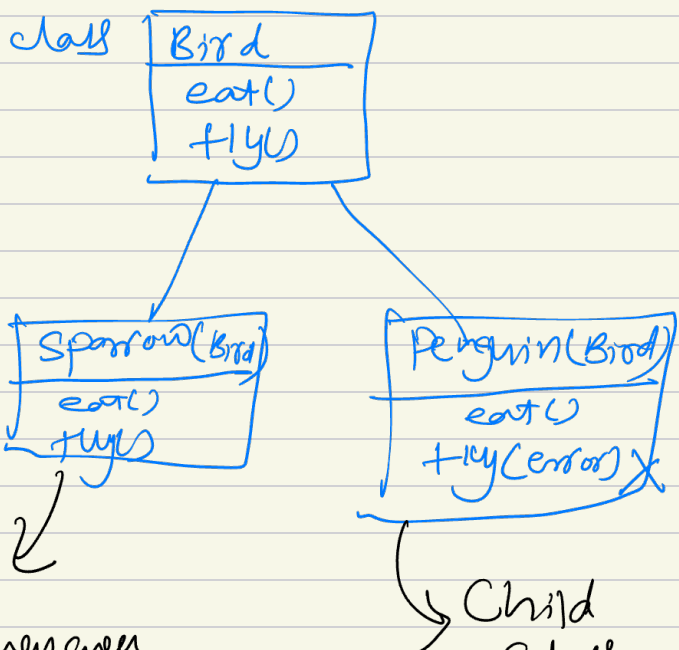
```
class AndSpecification(Specification):

    def __init__(self, *args):
        self.args = args

    def is_satisfied(self, item):
        return all(map(
                lambda spec:
                    spec.is_satisfied(item),
                    self.args))
```

Liskov substitution method

idea: if you have an interface derived from a base class you should be able to stick a derived class in it and everything should work.

- if child is derived from a parent class, then child should exactly work like parent

eg. Parent class



We should be able to use child class whenever Parent class is expected

Child class should not break the behaviour that parent class promises.

Bad example.

```
class BankC(ABC):          # → abstract class

    def __init__(self, amount):
        self.balance = amount

    @abstractmethod
    def deposit(self):
        pass

    @abstractmethod
    def withdraw(self):
        pass
```

```
class SavingsAccount(Bank):
    def __init__(self, amount):
        super().__init__(amount)

    def deposit(self, amount):
        self.balance += amount
        print(f"{self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print(error)
        else:
            self.balance -= amount
```

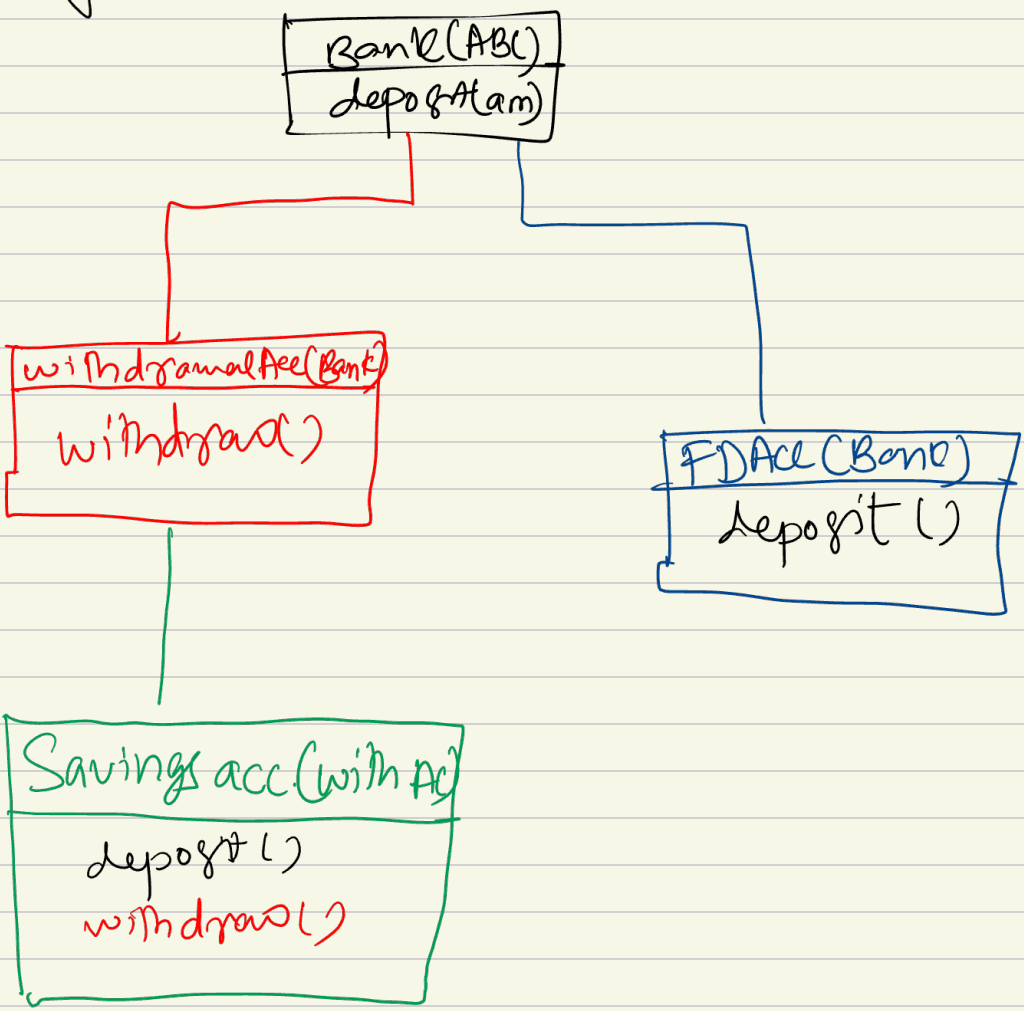
```
class FDAcc.(Bank):
    def __init__(self, amt)
        super().__init(amt)

    def deposit()
```

```
    def withdraw()
        raise Exception
```

here the pattern breaks

Solution to the bad example by LSP

Break the Base class into smaller class with seperation of problematic function



Interface Segregation Principle.

idea – "not to add too many methods inside a single interface"

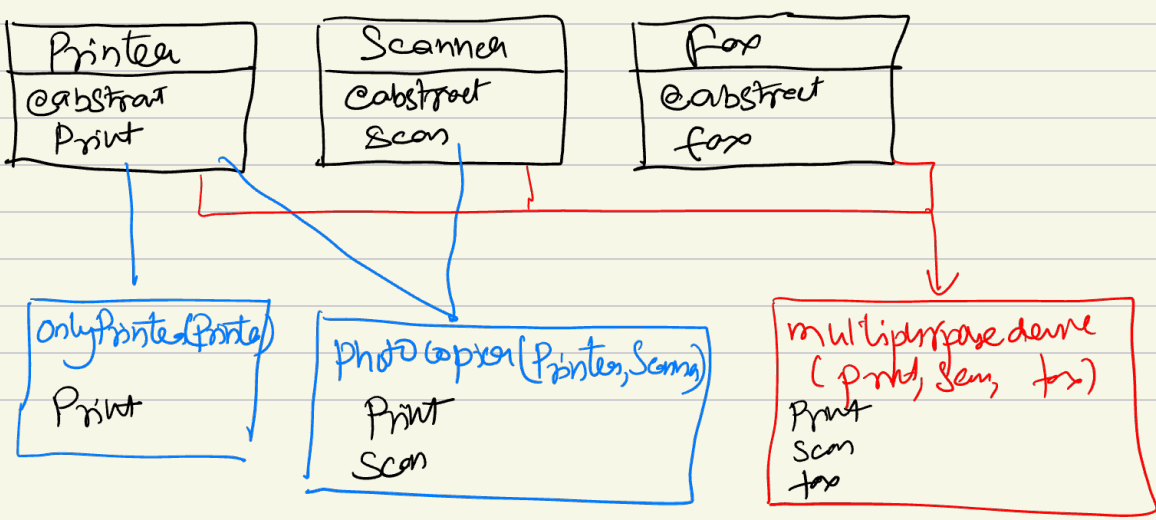
```
class machine:
    def print(self):
        pass
    def scan(self):
        pass
    def fax(self):
        pass
```

Printer: print ✓, scan ✓, fax ✗

OldPrinter: print ✓, scan ✗, fax ✓

all the inherited classes of machine class will have to have some definition for all the abstract functions.

Better approach.



Dependancy Inversion Principle.

idea - "High level classes/modules should not depend on low leved modules, instead they should depend on abstractions"

```
class Relation(Enum):
    CHILD=0
    PARENT=1
    SIBLING=2

class Person:
    def __init__(self, name):
        self.name=name

class Relationships:
    def __init__(self):
        self.relations=[]

    def add_parent_and_child(self,parent,child):
        self.relations.append(
            (parent, Relation.PARENT, child),
            (child, Relation.CHILD, parent))

class Research:
    def __init__(self,relationships):
        for r in relationships:
            if r[0].name=="Jt" and r[1]==relation.PARENT:
                print(f"Jt is parent of {r[2]}")
```

The main pain point here is that the Research class is dependent on the Relationships class's relation datastructure. What if it gets changed into dict, then the whole research class will fail to execute!

So, the Research class should depend on some sort of abstraction rather than the class itself (which will restructure underlying data structure change condition.

improved version.

```
class RelationshipBrowser:
    @abstractmethod
    def find_all_children_of(self, name): pass

class Relationships(RelationshipBrowser):
    def __init__(self):
        self.relations = []

    def add_parent_child(self, parent, child):
        self.relations.append(
            (parent, Relation.PARENT, child),
            (child, Relation.CHILD, parent)
        )

    def find_all_children_of(self, name):
        for r in self.relations:
            if r[0].name == name and r[1] == Relation.PARENT
                yield r[2].name.

class Research:
    def __init__(self, browser):
        for p in browser.find_all_children_of('lit'):
            print(f'Lit has a child called {p}')
```